CMPE 220 · SYSTEM SOFTWARE · SAN JOSÉ STATE UNIVERSITY · FALL 2026

CPU Design

Project Part 1 · Team 13 · the T13 16-bit CPU

Yashashav Devalapalli Kamalraj	`yashashav-dk`
Sai Karan Akula	`karanAkula`
Manoj Konda	`kmanoj03`

Code: `https://github.com/yashashav-dk/cmpe220-team13`

Built Nov 07, 2026 from the code by `make report`

How to run

Step	Command (repo root)	Expected
1. Get the code	`git clone https://github.com/yashashav-dk/cmpe220-team13.git` `cd cmpe220-team13`	needs a C99 compiler and make (macOS clang or Linux gcc)
2. Build, test, run every program	`make check`	last test line: `programs: all passed`; views in `build/views/`

3. Program A (8^2) by hand	`build/t13as programs/power.s -o power.img -l power.lst`
	`build/t13sim power.img -o power.trc`
	`build/t13view power.trc --waterfall`
		`power.img: HALT after 17 steps`; 18-row waterfall (§3)

4. Program B (8!)	`build/t13as programs/fact.s -o fact.img`
	`build/t13sim fact.img`
	`build/t13view fact.trc --peek result`
		`result=40320` (= `9D80`)

5. Same C, native	`cc programs/fact.c -o fact && ./fact`	`result=40320`

Table 1. how to download and run the code.

1 · CPU design

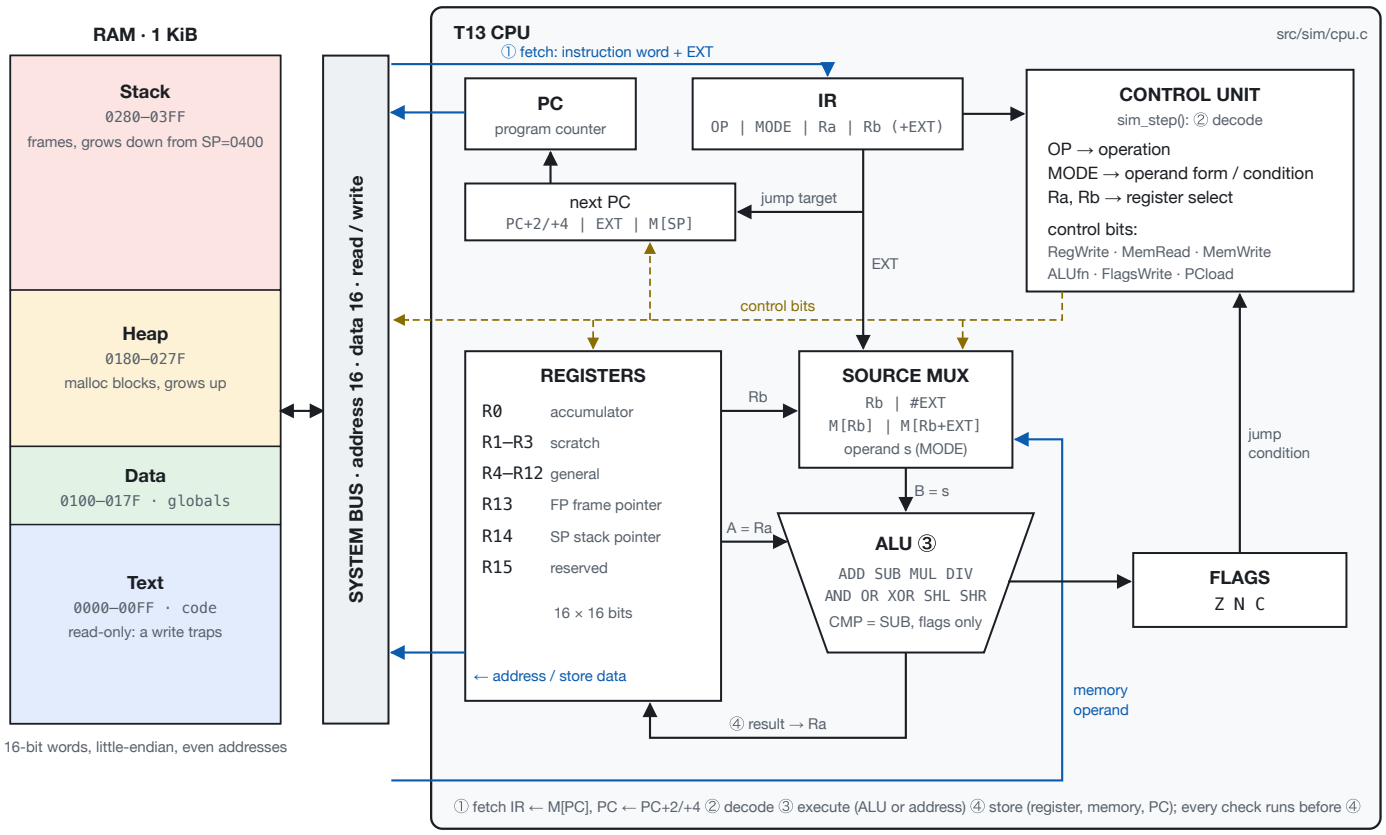


Figure 1. T13 CPU: datapath and control. Each block is a function in `src/sim/cpu.c`; ①–④ is one instruction cycle.

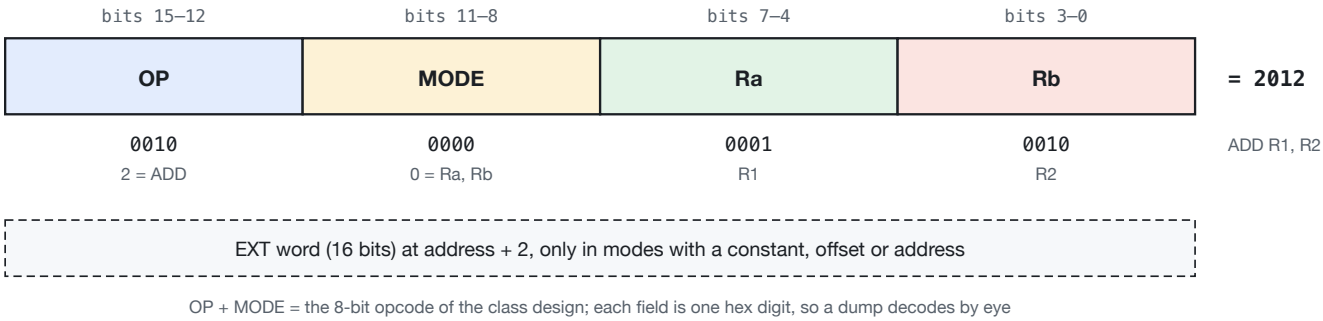


Figure 2. instruction word: 16 bits, four 4-bit fields, optional EXT word.

Register	Role
`R0`	accumulator, return value
R1–R3	scratch (caller-saved)
R4–R12	general (callee-saved)
`R13 = FP`	frame pointer
`R14 = SP`	stack pointer, starts at `0400`
`R15`	reserved
`PC · IR · FLAGS`	special: program counter, instruction register (word + EXT), Z N C

Table 2. registers.

Flag	Set by OP 2–A when
Z	result = 0
N	bit 15 of the result
C	ADD carry · SUB/CMP borrow · MUL product > FFFF · SHL/SHR last bit out · else 0
jumps	unsigned: LT = C, GE = ¬C, GT = ¬C∧¬Z, LE = C∨Z

Table 3. flags and jump conditions.

2 · Instruction encoding

OP	MODE	Opcode bits [15:8]	Assembly	Word(s)	Operation	Flags	Example	Hex (from t13as)
0	0	0000 0000	HALT	0000	stop the machine	–	HALT	**0000**
0	1	0000 0001	NOP	0100	no operation	–	NOP	**0100**
1	0	0001 0000	MOV Ra, Rb	10ab	Ra ← Rb	–	MOV R1, R2	**1012**
1	1	0001 0001	MOV Ra, #k	11a0 k	Ra ← k	–	MOV R1, #5	**1110 0005**
1	2	0001 0010	MOV Ra, [Rb]	12ab	Ra ← M[Rb]	–	MOV R0, [R2]	**1202**
1	3	0001 0011	MOV Ra, [Rb+k]	13ab k	Ra ← M[Rb+k]	–	MOV R3, [FP+4]	**133D 0004**
1	4	0001 0100	MOV [Ra], Rb	14ab	M[Ra] ← Rb	–	MOV [SP], R1	**14E1**
1	5	0001 0101	MOV [Ra+k], Rb	15ab k	M[Ra+k] ← Rb	–	MOV [FP+6], R0	**15D0 0006**
1	6	0001 0110	MOV Ra, [addr]	16a0 addr	Ra ← M[addr]	–	MOV R0, [0x0100]	**1600 0100**
1	7	0001 0111	MOV [addr], Ra	17a0 addr	M[addr] ← Ra	–	MOV [0x0102], R0	**1700 0102**
2	0	0010 0000	ADD Ra, Rb	20ab	Ra ← Ra + Rb	Z N C	ADD R1, R2	**2012**
2	1	0010 0001	ADD Ra, #k	21a0 k	Ra ← Ra + k	Z N C	ADD R1, #3	**2110 0003**
2	2	0010 0010	ADD Ra, [Rb]	22ab	Ra ← Ra + M[Rb]	Z N C	ADD R0, [R2]	**2202**
2	3	0010 0011	ADD Ra, [Rb+k]	23ab k	Ra ← Ra + M[Rb+k]	Z N C	ADD R0, [FP+4]	**230D 0004**
3	0	0011 0000	SUB Ra, Rb	30ab	Ra ← Ra − Rb	Z N C	SUB R1, R2	**3012**
3	1	0011 0001	SUB Ra, #k	31a0 k	Ra ← Ra − k	Z N C	SUB R1, #3	**3110 0003**
3	2	0011 0010	SUB Ra, [Rb]	32ab	Ra ← Ra − M[Rb]	Z N C	SUB R0, [R2]	**3202**
3	3	0011 0011	SUB Ra, [Rb+k]	33ab k	Ra ← Ra − M[Rb+k]	Z N C	SUB R0, [FP+4]	**330D 0004**
4	0	0100 0000	MUL Ra, Rb	40ab	Ra ← Ra × Rb	Z N C	MUL R1, R2	**4012**
4	1	0100 0001	MUL Ra, #k	41a0 k	Ra ← Ra × k	Z N C	MUL R1, #3	**4110 0003**
4	2	0100 0010	MUL Ra, [Rb]	42ab	Ra ← Ra × M[Rb]	Z N C	MUL R0, [R2]	**4202**
4	3	0100 0011	MUL Ra, [Rb+k]	43ab k	Ra ← Ra × M[Rb+k]	Z N C	MUL R0, [FP+4]	**430D 0004**
5	0	0101 0000	DIV Ra, Rb	50ab	Ra ← Ra ÷ Rb	Z N C	DIV R1, R2	**5012**
5	1	0101 0001	DIV Ra, #k	51a0 k	Ra ← Ra ÷ k	Z N C	DIV R1, #3	**5110 0003**
5	2	0101 0010	DIV Ra, [Rb]	52ab	Ra ← Ra ÷ M[Rb]	Z N C	DIV R0, [R2]	**5202**
5	3	0101 0011	DIV Ra, [Rb+k]	53ab k	Ra ← Ra ÷ M[Rb+k]	Z N C	DIV R0, [FP+4]	**530D 0004**
6	0	0110 0000	AND Ra, Rb	60ab	Ra ← Ra AND Rb	Z N C	AND R1, R2	**6012**
6	1	0110 0001	AND Ra, #k	61a0 k	Ra ← Ra AND k	Z N C	AND R1, #3	**6110 0003**
6	2	0110 0010	AND Ra, [Rb]	62ab	Ra ← Ra AND M[Rb]	Z N C	AND R0, [R2]	**6202**
6	3	0110 0011	AND Ra, [Rb+k]	63ab k	Ra ← Ra AND M[Rb+k]	Z N C	AND R0, [FP+4]	**630D 0004**
7	0	0111 0000	OR Ra, Rb	70ab	Ra ← Ra OR Rb	Z N C	OR R1, R2	**7012**
7	1	0111 0001	OR Ra, #k	71a0 k	Ra ← Ra OR k	Z N C	OR R1, #3	**7110 0003**
7	2	0111 0010	OR Ra, [Rb]	72ab	Ra ← Ra OR M[Rb]	Z N C	OR R0, [R2]	**7202**
7	3	0111 0011	OR Ra, [Rb+k]	73ab k	Ra ← Ra OR M[Rb+k]	Z N C	OR R0, [FP+4]	**730D 0004**
8	0	1000 0000	XOR Ra, Rb	80ab	Ra ← Ra XOR Rb	Z N C	XOR R1, R2	**8012**
8	1	1000 0001	XOR Ra, #k	81a0 k	Ra ← Ra XOR k	Z N C	XOR R1, #3	**8110 0003**
8	2	1000 0010	XOR Ra, [Rb]	82ab	Ra ← Ra XOR M[Rb]	Z N C	XOR R0, [R2]	**8202**
8	3	1000 0011	XOR Ra, [Rb+k]	83ab k	Ra ← Ra XOR M[Rb+k]	Z N C	XOR R0, [FP+4]	**830D 0004**
9	0	1001 0000	SHL Ra, Rb	90ab	Ra ← Ra << (Rb & 15)	Z N C	SHL R1, R2	**9012**
9	1	1001 0001	SHL Ra, #n	91a0 n	Ra ← Ra << n	Z N C	SHL R1, #1	**9110 0001**
9	2	1001 0010	SHR Ra, Rb	92ab	Ra ← Ra >> (Rb & 15)	Z N C	SHR R1, R2	**9212**
9	3	1001 0011	SHR Ra, #n	93a0 n	Ra ← Ra >> n (logical)	Z N C	SHR R1, #1	**9310 0001**
A	0	1010 0000	CMP Ra, Rb	A0ab	flags of Ra − Rb	Z N C	CMP R1, R2	**A012**
A	1	1010 0001	CMP Ra, #k	A1a0 k	flags of Ra − k	Z N C	CMP R1, #0	**A110 0000**
B	0	1011 0000	JMP addr	B000 addr	PC ← addr	–	JMP 0x0010	**B000 0010**
B	1	1011 0001	JEQ addr	B100 addr	if Z: PC ← addr	–	JEQ 0x0010	**B100 0010**
B	2	1011 0010	JNE addr	B200 addr	if not Z	–	JNE 0x0010	**B200 0010**
B	3	1011 0011	JLT addr	B300 addr	if C (unsigned <)	–	JLT 0x0010	**B300 0010**
B	4	1011 0100	JGE addr	B400 addr	if not C	–	JGE 0x0010	**B400 0010**
B	5	1011 0101	JGT addr	B500 addr	if not C and not Z	–	JGT 0x0010	**B500 0010**
B	6	1011 0110	JLE addr	B600 addr	if C or Z	–	JLE 0x0010	**B600 0010**
C	0	1100 0000	CALL addr	C000 addr	SP ← SP−2; M[SP] ← PC; PC ← addr	–	CALL 0x0040	**C000 0040**
D	0	1101 0000	RET	D000	PC ← M[SP]; SP ← SP+2	–	RET	**D000**

Table 4. instruction encoding, every legal OP/MODE pair (53). a = Ra field, b = Rb field, k / n / addr = the EXT word that follows; M[x] = 16-bit word at x. Hex column assembled by `t13as` and checked against `include/isa_ops.def`. Codes E and F, and any MODE not listed, are illegal instructions.

3 · Program A: 8^2 written in T13 assembly

Addr	Words	Label	Assembly	Comment
			`.equ BASE, 8`	`compute BASE^EXP = 8^2`
			`.equ EXP, 2`	
			`.text`	
`0000`	`1100 0001`	`main:`	`MOV R0, #1`	`R0 = result, starts at 1`
`0004`	`1110 0008`		`MOV R1, #BASE`	`R1 = base`
`0008`	`1120 0002`		`MOV R2, #EXP`	`R2 = exponent left`
`000C`	`A120 0000`	`loop:`	`CMP R2, #0`	`done when the exponent reaches 0`
`0010`	`B100 001E`		`JEQ done`	
`0014`	`4001`		`MUL R0, R1`	`result *= base`
`0016`	`3120 0001`		`SUB R2, #1`	`exponent -= 1`
`001A`	`B000 000C`		`JMP loop`	
`001E`	`1700 0100`	`done:`	`MOV [result], R0`	`store the answer in the Data word`
`0022`	`0000`		`HALT`	
			`.data`	
`0100`	`0000`	`result:`	`.word 0`	`becomes 0x0040 = 64`

Table 5. `programs/power.s`: $\text{BASE}^{\text{EXP}} = 8^2$ by repeated MUL; addresses and words from `t13as`.

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Table 6. `power.s`: memory after loading (step 0); rows with code, data or stack use.

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	**0040** result	0000	0000	0000	0000	0000	0000	0000

Table 7. `power.s`: memory at the end (step 17, END HALT); shaded = changed since loading.

Program A: memory after every instruction

Step	PC	Instruction	R0	R1	R2	Z	N	C	0100 result
0	–	(loaded)	0000	0000	0000	0	0	0	0000
1	0000	MOV R0, #1	**0001**	0000	0000	0	0	0	0000
2	0004	MOV R1, #8	0001	**0008**	0000	0	0	0	0000
3	0008	MOV R2, #2	0001	0008	**0002**	0	0	0	0000
4	000C	CMP R2, #0	0001	0008	0002	0	0	0	0000
5	0010	JEQ 0x001E	0001	0008	0002	0	0	0	0000
6	0014	MUL R0, R1	**0008**	0008	0002	0	0	0	0000
7	0016	SUB R2, #1	0008	0008	**0001**	0	0	0	0000
8	001A	JMP 0x000C	0008	0008	0001	0	0	0	0000
9	000C	CMP R2, #0	0008	0008	0001	0	0	0	0000
10	0010	JEQ 0x001E	0008	0008	0001	0	0	0	0000
11	0014	MUL R0, R1	**0040**	0008	0001	0	0	0	0000
12	0016	SUB R2, #1	0040	0008	**0000**	**1**	0	0	0000
13	001A	JMP 0x000C	0040	0008	0000	1	0	0	0000
14	000C	CMP R2, #0	0040	0008	0000	1	0	0	0000
15	0010	JEQ 0x001E	0040	0008	0000	1	0	0	0000
16	001E	MOV [0x0100], R0	0040	0008	0000	1	0	0	**0040**
17	0022	HALT	0040	0008	0000	1	0	0	0040

Table 8. `power.s`: memory waterfall, one row per executed instruction (row 0: after loading). Columns: registers and memory words that change; shaded = changed by that row's instruction.

4 · Program B: 8! written in C, translated to T13 assembly

C (programs/*.c)	Addr	Words	T13 assembly (programs/*.s)	Comment
`int main(void) {`				
`result = fact(8);`	`0000`	`31E0 0002`	`main:  SUB SP, #2`	`reserve the return-value slot`
	`0004`	`1110 0008`	`MOV R1, #8`	
	`0008`	`31E0 0002 14E1`	`PUSH R1`	`argument n = 8`
	`000E`	`C000 0022`	`CALL fact`	
	`0012`	`21E0 0002`	`ADD SP, #2`	`drop the argument`
	`0016`	`120E 21E0 0002`	`POP R0`	`R0 = fact(8)`
	`001C`	`1700 0100`	`MOV [result], R0`	
`printf(...);  (not translated: no I/O)`				
`return 0;`	`0020`	`0000`	`HALT`	
`}`				
`uint16_t fact(uint16_t n) {`	`0022`	`31E0 0002 14ED`	`fact:  PUSH FP`	`prologue: save the caller's FP`
	`0028`	`10DE`	`MOV FP, SP`	`FP -> this frame`
`uint16_t r = 1;`	`002A`	`1100 0001`	`MOV R0, #1`	`r lives in R0`
	`002E`	`131D 0004`	`MOV R1, [FP+ARG]`	`n lives in R1`
`while (n > 1) {`	`0032`	`A110 0001`	`loop:  CMP R1, #1`	
	`0036`	`B600 0044`	`JLE done`	`unsigned n <= 1: leave the loop`
`r = r * n;`	`003A`	`4001`	`MUL R0, R1`	
`n = n - 1;`	`003C`	`3110 0001`	`SUB R1, #1`	
	`0040`	`B000 0032`	`JMP loop`	
`}`				
`return r;`	`0044`	`15D0 0006`	`done:  MOV [FP+RETV], R0`	`result into the caller's slot`
	`0048`	`10ED`	`MOV SP, FP`	`epilogue`
	`004A`	`12DE 21E0 0002`	`POP FP`	
	`0050`	`D000`	`RET`	
`}`				
`(Data segment)`	`0100`	`0000`	`result:  .word 0`	`becomes 0x9D80 = 40320`

Table 9. `programs/fact.c` next to `programs/fact.s`: each C line and the instructions that implement it.

Program B: memory after loading and at the end

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	31E0 main	0002	1110	0008	31E0	0002	14E1	C000
TEXT	0010	0022	21E0	0002	120E	21E0	0002	1700	0100
TEXT	0020	0000	31E0 fact	0002	14ED	10DE	1100	0001	131D
TEXT	0030	0004	A110 loop	0001	B600	0044	4001	3110	0001
TEXT	0040	B000	0032	15D0 done	0006	10ED	12DE	21E0	0002
TEXT	0050	D000	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000
STACK	03F0	0000	0000	0000	0000	0000	0000	0000	0000

Table 10. `fact.s`: memory after loading (step 0); rows with code, data or stack use.

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	31E0 main	0002	1110	0008	31E0	0002	14E1	C000
TEXT	0010	0022	21E0	0002	120E	21E0	0002	1700	0100
TEXT	0020	0000	31E0 fact	0002	14ED	10DE	1100	0001	131D
TEXT	0030	0004	A110 loop	0001	B600	0044	4001	3110	0001
TEXT	0040	B000	0032	15D0 done	0006	10ED	12DE	21E0	0002
TEXT	0050	D000	0000	0000	0000	0000	0000	0000	0000
DATA	0100	**9D80** result	0000	0000	0000	0000	0000	0000	0000
STACK	03F0	0000	0000	0000	0000	0000	**0012**	**0008**	**9D80**

Table 11. `fact.s`: memory at the end (step 57, END HALT); shaded = changed since loading.

Program B: memory after every instruction

Step	PC	Instruction	R0	R1	FP	SP	Z	N	C	0100 result	03F8	03FA	03FC	03FE
0	—	(loaded)	0000	0000	0000	0400	0	0	0	0000	0000	0000	0000	0000
1	0000	SUB SP, #2	0000	0000	0000	**03FE**	0	0	0	0000	0000	0000	0000	0000
2	0004	MOV R1, #8	0000	**0008**	0000	03FE	0	0	0	0000	0000	0000	0000	0000
3	0008	SUB SP, #2	0000	0008	0000	**03FC**	0	0	0	0000	0000	0000	0000	0000
4	000C	MOV [SP], R1	0000	0008	0000	03FC	0	0	0	0000	0000	0000	**0008**	0000
5	000E	CALL 0x0022	0000	0008	0000	**03FA**	0	0	0	0000	0000	**0012**	0008	0000
6	0022	SUB SP, #2	0000	0008	0000	**03F8**	0	0	0	0000	0000	0012	0008	0000
7	0026	MOV [SP], FP	0000	0008	0000	03F8	0	0	0	0000	0000	0012	0008	0000
8	0028	MOV FP, SP	0000	0008	**03F8**	03F8	0	0	0	0000	0000	0012	0008	0000
9	002A	MOV R0, #1	**0001**	0008	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
10	002E	MOV R1, [FP+4]	0001	0008	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
11	0032	CMP R1, #1	0001	0008	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
12	0036	JLE 0x0044	0001	0008	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
13	003A	MUL R0, R1	**0008**	0008	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
14	003C	SUB R1, #1	0008	**0007**	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
15	0040	JMP 0x0032	0008	0007	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
16	0032	CMP R1, #1	0008	0007	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
17	0036	JLE 0x0044	0008	0007	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
18	003A	MUL R0, R1	**0038**	0007	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
19	003C	SUB R1, #1	0038	**0006**	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
20	0040	JMP 0x0032	0038	0006	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
21	0032	CMP R1, #1	0038	0006	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
22	0036	JLE 0x0044	0038	0006	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
23	003A	MUL R0, R1	**0150**	0006	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
24	003C	SUB R1, #1	0150	**0005**	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
25	0040	JMP 0x0032	0150	0005	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
26	0032	CMP R1, #1	0150	0005	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
27	0036	JLE 0x0044	0150	0005	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
28	003A	MUL R0, R1	**0690**	0005	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
29	003C	SUB R1, #1	0690	**0004**	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
30	0040	JMP 0x0032	0690	0004	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
31	0032	CMP R1, #1	0690	0004	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
32	0036	JLE 0x0044	0690	0004	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
33	003A	MUL R0, R1	**1A40**	0004	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
34	003C	SUB R1, #1	1A40	**0003**	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
35	0040	JMP 0x0032	1A40	0003	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
36	0032	CMP R1, #1	1A40	0003	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
37	0036	JLE 0x0044	1A40	0003	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
38	003A	MUL R0, R1	**4EC0**	0003	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
39	003C	SUB R1, #1	4EC0	**0002**	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
40	0040	JMP 0x0032	4EC0	0002	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
41	0032	CMP R1, #1	4EC0	0002	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
42	0036	JLE 0x0044	4EC0	0002	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
43	003A	MUL R0, R1	**9D80**	0002	03F8	03F8	0	**1**	0	0000	0000	0012	0008	0000
44	003C	SUB R1, #1	9D80	**0001**	03F8	03F8	0	**0**	0	0000	0000	0012	0008	0000
45	0040	JMP 0x0032	9D80	0001	03F8	03F8	0	0	0	0000	0000	0012	0008	0000
46	0032	CMP R1, #1	9D80	0001	03F8	03F8	**1**	0	0	0000	0000	0012	0008	0000
47	0036	JLE 0x0044	9D80	0001	03F8	03F8	1	0	0	0000	0000	0012	0008	0000
48	0044	MOV [FP+6], R0	9D80	0001	03F8	03F8	1	0	0	0000	0000	0012	0008	**9D80**
49	0048	MOV SP, FP	9D80	0001	03F8	03F8	1	0	0	0000	0000	0012	0008	9D80
50	004A	MOV FP, [SP]	9D80	0001	**0000**	03F8	1	0	0	0000	0000	0012	0008	9D80
51	004C	ADD SP, #2	9D80	0001	0000	**03FA**	**0**	0	0	0000	0000	0012	0008	9D80
52	0050	RET	9D80	0001	0000	**03FC**	0	0	0	0000	0000	0012	0008	9D80
53	0012	ADD SP, #2	9D80	0001	0000	**03FE**	0	0	0	0000	0000	0012	0008	9D80
54	0016	MOV R0, [SP]	9D80	0001	0000	03FE	0	0	0	0000	0000	0012	0008	9D80
55	0018	ADD SP, #2	9D80	0001	0000	**0400**	0	0	0	0000	0000	0012	0008	9D80
56	001C	MOV [0x0100], R0	9D80	0001	0000	0400	0	0	0	**9D80**	0000	0012	0008	9D80
57	0020	HALT	9D80	0001	0000	0400	0	0	0	9D80	0000	0012	0008	9D80

Table 12. `fact.s`: memory waterfall, one row per executed instruction (row 0: after loading). Columns: registers and memory words that change; shaded = changed by that row's instruction.

5 · Tests and contributions

Suite	Checks	Result (this build)
`test_isa`	codec: encode / decode / disassemble every instruction	`561 checks, 0 failed`
`test_asm`	assembler parser, symbols, instruction selection, disasm → asm round trip	`773 checks, 0 failed`
`test_assemble`	two-pass assembler, directives, errors, image and listing writers	`158 checks, 0 failed`
`test_sim`	simulator: loader errors, every ALU flag rule, jumps, CALL/RET, faults, trace	`284 checks, 0 failed`
`test_view`	viewer: trace checks (rule 11), every view	`120 checks, 0 failed`
`tests/golden.sh`	t13as images and listings, t13sim traces vs hand-written golden traces, CLI exit codes	`golden: all passed`

`tests/programs.sh`	every program end to end; each .c compiled natively prints what t13sim leaves in memory	`ok power.s: result=64 after 17 steps`
		`ok fact.s: result=40320 after 57 steps`
		`ok fact.c: native cc prints result=40320`
		`ok recfact.s: *result=120 after 112 steps`
		`ok recfact.c: native cc prints *result=120`
		`programs: all passed`

Table 13. test results, captured while building this PDF (`make test`).

Member	Code and documents	Part 1 report	Part 2 report
Yashashav Devalapalli Kamalraj	Repository and MIT license. ISA v1.0 `docs/isa.md`, opcode table `isa_ops.def`, codec `isa.c` with 561 checks; assembler `t13as` with listing and manual `docs/asm.md`, 931 checks and golden files; Program A: 8^2 in T13 assembly.	§2 instruction encoding table; §3 Program A.	§3 executable loaded into memory; put the Part 2 PDF together.
Sai Karan Akula	Reviewed and approved ISA v1.0 (Nov 3). Image format `docs/image.md`, stack-frame layout `docs/abi.md`, simulator `t13sim` (loader and CPU), Program B: 8! from C to T13 assembly.	§1 CPU diagram; §4 Program B; put the Part 1 PDF together.	§1 program-layout and stack-frame diagrams; §2 calling sequence; §4 CALL/RET snapshots and stack checks.
Manoj Konda	Trace format `docs/trace.md` and the hand-written golden traces `halt.trc` and `power.trc` (the simulator's acceptance tests); CI on Linux gcc/clang and macOS clang (PR #3). Repository layout and `Makefile`, viewer `t13view`, the recursive Part 2 program `recfact`.	§3 and §4 memory tables and their legend, the appendix; title page and page-number template.	§2 `recfact` C next to assembly; §4 four-segment view with stack frames, one snapshot per call and return.

Table 14. contributions of each member.

Appendix · Program A, full memory block after every instruction

Step 1: `MOV R0, #1`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 2: `MOV R1, #8`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 3: `MOV R2, #2`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 4: `CMP R2, #0`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 5: `JEQ 0x001E`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 6: `MUL R0, R1`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 7: `SUB R2, #1`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 8: `JMP 0x000C`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 9: `CMP R2, #0`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 10: `JEQ 0x001E`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 11: `MUL R0, R1`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 12: `SUB R2, #1`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 13: `JMP 0x000C`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 14: `CMP R2, #0`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 15: `JEQ 0x001E`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0000 result	0000	0000	0000	0000	0000	0000	0000

Step 16: `MOV [0x0100], R0`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	**0040** result	0000	0000	0000	0000	0000	0000	0000

Step 17: `HALT`

Seg	Addr	+0	+2	+4	+6	+8	+A	+C	+E
TEXT	0000	1100 main	0001	1110	0008	1120	0002	A120 loop	0000
TEXT	0010	B100	001E	4001	3120	0001	B000	000C	1700 done
TEXT	0020	0100	0000	0000	0000	0000	0000	0000	0000
DATA	0100	0040 result	0000	0000	0000	0000	0000	0000	0000

Table 15. `power.s`: the memory block after every executed instruction; shaded = written by that instruction.